

Streamlit Cheatsheet (from Advanced Stock Analyzer)

1. Page Configuration

```
st.set_page_config(
    page_title="Advanced Stock Analyzer",
    layout="wide",          # "centered" | "wide"
    initial_sidebar_state="expanded"  # "auto" | "expanded" | "collapsed"
)
```

2. Sidebar Widgets

```
st.sidebar.header("Stock Selection")

# Text input
ticker = st.sidebar.text_input("Enter Stock Symbol", value="GOOGL").upper()

# Columns inside sidebar
col1, col2 = st.sidebar.columns(2)
with col1:
    start_date = st.date_input("Start Date", value=datetime.now() - timedelta(days=365*10))
with col2:
    end_date = st.date_input("End Date", value=datetime.now())

# Checkbox
show_ma = st.sidebar.checkbox("Show Moving Averages", value=True)

# Multiselect
ma_periods = st.sidebar.multiselect(
    "MA Periods",
    options=[10, 20, 50, 100, 200],
    default=[20, 50]
)

# Number input
risk_free_rate = st.sidebar.number_input(
    "Risk-Free Rate (annual, %)",
    min_value=0.0, max_value=20.0, value=2.0, step=0.25
) / 100
```

3. Layout & Containers

```
# Markdown with HTML
st.markdown(f'<h1 class="main-header">{tickerSymbol}</h1>', unsafe_allow_html=True)

# Columns (equal width)
col1, col2, col3, col4 = st.columns(4)

# Column with content
with col1:
    st.metric("Company", info.get('shortName', tickerSymbol)[:30] + '...')

# Divider
st.divider()

# Subheader
st.subheader("Key Statistics")
```

4. Metrics

```
# Simple metric
st.metric("Volume", f"{data['Volume'].iloc[-1]:,}.0f")

# Metric with delta (green/red arrow)
```

```

st.metric(
    "Current Price",
    f"${current_price:.2f}",
    f"{change:.2f}"          # delta
)

```

5. Caching (Critical for Performance)

```

@st.cache_data(ttl=3600)      # ttl in seconds
def fetch_stock_data(ticker, start, end):
    stock = yf.Ticker(ticker)
    data = stock.history(start=start, end=end)
    return data, stock.info

# Cache with try/except
@st.cache_data(ttl=3600)
def fetch_comparison_data(ticker, start, end):
    try:
        stock = yf.Ticker(ticker)
        return stock.history(start=start, end=end)
    except:
        return None

```

6. Error Handling & Feedback

```

# Error message
st.error(f"Error fetching data: {e}")

# Warning
st.warning("Short MA window should be smaller than Long MA window.")

# Caption (small text)
st.caption("Backtest assumes no transaction costs. Not investment advice.")

```

7. Charts (Plotly Integration)

```

import plotly.graph_objects as go
from plotly.subplots import make_subplots

# Basic chart
fig = go.Figure()
fig.add_trace(go.Scatter(x=data.index, y=data['Close'], mode='lines', name='Price'))

# Subplots with shared x-axis
fig = make_subplots(
    rows=n_rows, cols=1,
    shared_xaxes=True,
    vertical_spacing=0.08,
    row_heights=[0.5, 0.15, 0.175, 0.175][:n_rows],
    subplot_titles=subtitles
)

# Add trace to specific subplot
fig.add_trace(
    go.Scatter(x=data.index, y=data['RSI'], name='RSI'),
    row=row_counter, col=1
)

# Horizontal lines (support/resistance)
fig.add_hline(y=70, line_dash='dash', line_color='red', row=row_counter, col=1)
fig.add_hline(y=30, line_dash='dash', line_color='green', row=row_counter, col=1)

# Horizontal rectangles (zones)
fig.add_hrect(y0=70, y1=100, line_dash='dash', line_color='red', opacity=0.1, row=row_counter, col=1)

# Bar chart (volume)
fig.add_trace(
    go.Bar(x=data.index, y=data['Volume'], name='Volume', marker=dict(color=vol_colors)),
    row=row_counter, col=1
)

```

```

)

# Update layout
fig.update_layout(
    height=200 + (n_rows * 200),
    showlegend=True,
    hovermode='x unified',
    template='plotly_white',
    legend=dict(orientation='h', yanchor='bottom', y=1.02, xanchor='right', x=1)
)

fig.update_xaxes(title_text='Date', row=n_rows, col=1)
fig.update_yaxes(title_text='Price ($)', row=n_rows, col=1)

# Render in Streamlit
st.plotly_chart(fig, use_container_width=True)

```

8. Data Download

```

csv = data.to_csv()
st.download_button(
    label="Download CSV",
    data=csv,
    file_name=f"{tickerSymbol}_data.csv",
    mime="text/csv"
)

```

9. Common Patterns

Conditional Rendering

```

if show_rsi:
    data['RSI'] = ta.momentum.RSIIndicator(data['Close'], window=14).rsi()
    # ... add to chart

```

Handling Empty/None Data

```

data, info = fetch_stock_data(tickerSymbol, start_date, end_date)

if data is not None and not data.empty:
    # ... main app logic
else:
    st.error(f"Could not fetch data for {tickerSymbol}.")

```

Dynamic Row Counting

```

n_rows = 2 # price + volume
if show_rsi:
    n_rows += 1
if show_macd:
    n_rows += 1

```

Color-Coded Bars

```

vol_colors = []
for i in range(len(data)):
    if i == 0 or data['Close'].iloc[i] >= data['Close'].iloc[i-1]:
        vol_colors.append('green')
    else:
        vol_colors.append('red')

```

F-String Formatting in Metrics

```

f"${value:.2f}"          # 2 decimals with $
f"${value:,.0f}"         # thousands separator, no decimals

```

```
f"{value * 100:.2f}%" # percentage
f"{value:.2f}"        # plain 2 decimals
```

10. Quick Reference Table

Task	Code
Page config	<code>st.set_page_config(page_title, layout, initial_sidebar_state)</code>
Text input	<code>st.text_input(label, value)</code>
Number input	<code>st.number_input(label, min_value, max_value, value, step)</code>
Date input	<code>st.date_input(label, value)</code>
Checkbox	<code>st.checkbox(label, value)</code>
Multiselect	<code>st.multiselect(label, options, default)</code>
Columns	<code>st.columns(n)</code>
Metric	<code>st.metric(label, value, delta)</code>
Divider	<code>st.divider()</code>
Subheader	<code>st.subheader(text)</code>
Markdown	<code>st.markdown(text, unsafe_allow_html)</code>
Error	<code>st.error(msg)</code>
Warning	<code>st.warning(msg)</code>
Caption	<code>st.caption(text)</code>
Cache	<code>@st.cache_data(ttl=seconds)</code>
Plotly	<code>st.plotly_chart(fig, use_container_width=True)</code>
Download	<code>st.download_button(label, data, file_name, mime)</code>
Sidebar	<code>st.sidebar.<widget></code>

11. Important Notes

- `@st.cache_data` is for functions returning data (DataFrames, dicts). Use `ttl` to auto-expire.
- `unsafe_allow_html=True` is required for HTML in `st.markdown`.
- `use_container_width=True` makes charts responsive.
- `shared_xaxes=True` in `make_subplots` links zoom/pan across subplots.
- **F-strings inside metrics** must escape quotes carefully: `f"{risk["CAGR"] * 100:.2f}"` works in Python 3.12+; use single quotes outside for older versions: `f'{risk["CAGR"] * 100:.2f}'`.
- **Sidebar columns** use `st.sidebar.columns(n)`, not `st.columns` then `st.sidebar`.